LAPORAN TUGAS

LATIHAN TERSTRUKTUR

PERTEMUAN 4: IMPLEMENTASI UNIT TESTING DENGAN GITHUB ACTIONS

Mata Kuliah: Implementasi Pengujian Perangkat Lunak



Disusun oleh:

Nama	:	Chepi syahbudien basil
NIM	:	1224022
Kelas	:	Reguler pagi

BAGIAN A: GIT & GITHUB DASAR

Soal 1: Konsep Version Control System

1. Perbedaan Centralized VCS dan Distributed VCS

Aspek	Centralized VCS (CVCS)	Distributed VCS (DVCS)
Penyimpanan repository	Satu repository pusat di server; developer hanya menyalin versi kerja (working copy).	Setiap developer memiliki salinan lengkap repository beserta seluruh riwayatnya.
Kerja offline	Sulit; commit dan melihat riwayat membutuhkan koneksi ke server.	Bisa; commit, branch, dan melihat riwayat dilakukan lokal tanpa internet.
Risiko kegagalan	Single point of failure: jika server rusak, riwayat bisa hilang.	Setiap salinan lokal adalah backup lengkap.
Branching/merging	Relatif berat dan lambat.	Ringan dan cepat.
Contoh	1. CVS 2. Apache Subversion (SVN)	1. Git 2. Mercurial

2. Lima Git command yang wajib dikuasai

No	Command	Fungsi
1	git clone <url>	Menyalin repository dari remote (misalnya GitHub) ke komputer lokal.
2	git add <file>	Memasukkan perubahan file ke staging area sebelum di-commit.
3	git commit -m "pesan"	Menyimpan perubahan yang ada di staging area sebagai satu snapshot (commit) di riwayat lokal.
4	git push <remote> <branch>	Mengirim commit lokal ke repository remote.
5	git pull	Mengambil dan menggabungkan perubahan terbaru dari remote ke repository lokal.

3. Fungsi .gitignore

File .gitignore berisi daftar pola nama file atau folder yang tidak boleh dilacak oleh Git. Fungsinya adalah menjaga repository tetap bersih dan aman, misalnya dengan mengecualikan hasil kompilasi (target/, *.class), file konfigurasi IDE (.idea/, .vscode/), dependensi yang besar (node_modules/), serta file rahasia seperti .env atau kunci API yang tidak boleh terunggah ke GitHub.

Soal 2: Praktik Git Commands

Ganti Chepi-dev , 1224022, dan Chepi syahbudien basil dengan data Anda. Repository dibuat kosong (tanpa README otomatis) di langkah 1.

No	Langkah	Command
1	Buat repository baru di GitHub	Membuat Repositori dengan nama Latihan-git-1224022
2	Clone ke lokal	git clone https://github.com/chepi-dev/latihan-git-1224022.git
3	Buat README.md berisi nama dan NIM	echo "# latihan-git-1224022" >> README.md echo "Nama: Chepi syahbudien basil" >> README.md echo "NIM: 1224022" >> README.md
4	Commit README	git add README.md git commit -m "first commit"
5	Push ke GitHub	git branch -M main git push -u origin main
6	Buat branch feature-test	git branch feature-test
7	Pindah ke branch tersebut	git switch feature-test
8	Buat test.txt dan commit	echo "file uji coba" > test.txt git add test.txt git commit -m "test: add test file"
9	Push branch feature-test	git push -u origin feature-test
10	Merge feature-test ke main	git switch main git merge feature-test git push origin main

BAGIAN B: UNIT TESTING FRAMEWORK

Soal 3: Konversi Test Case ke Kode Unit Test

Method validasiPendaftaran dari Tugas Minggu 3 memiliki 7 independent path (V(G) = 7). Soal meminta 5 test case; kelima test case utama (TC-1 sampai TC-5) mewakili setiap kemungkinan nilai return, sedangkan TC-6 dan TC-7 adalah tambahan agar seluruh 7 independent path tercakup.

1. Test Case berdasarkan Independent Path

TC	Path	Nama	Umur	Email	Setuju	Expected Output
TC-1	1	null	20	"budi@mail.com"	true	"Nama harus diisi"
TC-2	3	"Budi"	16	"budi@mail.com"	true	"Umur minimal 17 tahun"
TC-3	5	"Budi"	20	"budimail.com"	true	"Email tidak valid"
TC-4	6	"Budi"	20	"budi@mail.com"	false	"Harus menyetujui syarat dan ketentuan"
TC-5	7	"Budi"	20	"budi@mail.com"	true	"Pendaftaran berhasil"
TC-6*	2	""	20	"budi@mail.com"	true	"Nama harus diisi"
TC-7*	4	"Budi"	20	null	true	"Email tidak valid"

* test case tambahan untuk melengkapi 7 independent path.

2. Implementasi JUnit 5

Kode yang diuji (PendaftaranService.java):

```
public class PendaftaranService {

    public String validasiPendaftaran(String nama, int umur, String email,
                                      boolean setujuSyarat) {
        if (nama == null || nama.isEmpty()) {
            return "Nama harus diisi";
        }
        if (umur < 17) {
            return "Umur minimal 17 tahun";
        }
        if (email == null || !email.contains("@")) {
            return "Email tidak valid";
        }
        if (!setujuSyarat) {
            return "Harus menyetujui syarat dan ketentuan";
        }
        return "Pendaftaran berhasil";
    }
}
```

Kode unit test (PendaftaranServiceTest.java):

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
```

```
class PendaftaranServiceTest {

    private PendaftaranService service;

    @BeforeEach
    void setUp() {
        service = new PendaftaranService();
    }

    @Test
    @DisplayName("TC-1 (Path 1): nama null ditolak")
    void namaNull() {
        assertEquals("Nama harus diisi",
                service.validasiPendaftaran(null, 20, "budi@mail.com", true));
    }

    @Test
    @DisplayName("TC-2 (Path 3): umur di bawah 17 ditolak")
    void umurDiBawah17() {
        assertEquals("Umur minimal 17 tahun",
                service.validasiPendaftaran("Budi", 16, "budi@mail.com", true));
    }

    @Test
    @DisplayName("TC-3 (Path 5): email tanpa '@' ditolak")
    void emailTanpaAt() {
        assertEquals("Email tidak valid",
                service.validasiPendaftaran("Budi", 20, "budimail.com", true));
    }

    @Test
    @DisplayName("TC-4 (Path 6): syarat tidak disetujui ditolak")
    void syaratTidakDisetujui() {
        assertEquals("Harus menyetujui syarat dan ketentuan",
                service.validasiPendaftaran("Budi", 20, "budi@mail.com", false));
    }

    @Test
    @DisplayName("TC-5 (Path 7): semua data valid, pendaftaran berhasil")
    void semuaValid() {
        assertEquals("Pendaftaran berhasil",
                service.validasiPendaftaran("Budi", 20, "budi@mail.com", true));
    }

    // ---- Test case tambahan agar 7 independent path terpenuhi ----

    @Test
    @DisplayName("TC-6 (Path 2): nama kosong ditolak")
    void namaKosong() {
        assertEquals("Nama harus diisi",
                service.validasiPendaftaran("", 20, "budi@mail.com", true));
    }

    @Test
    @DisplayName("TC-7 (Path 4): email null ditolak")
```

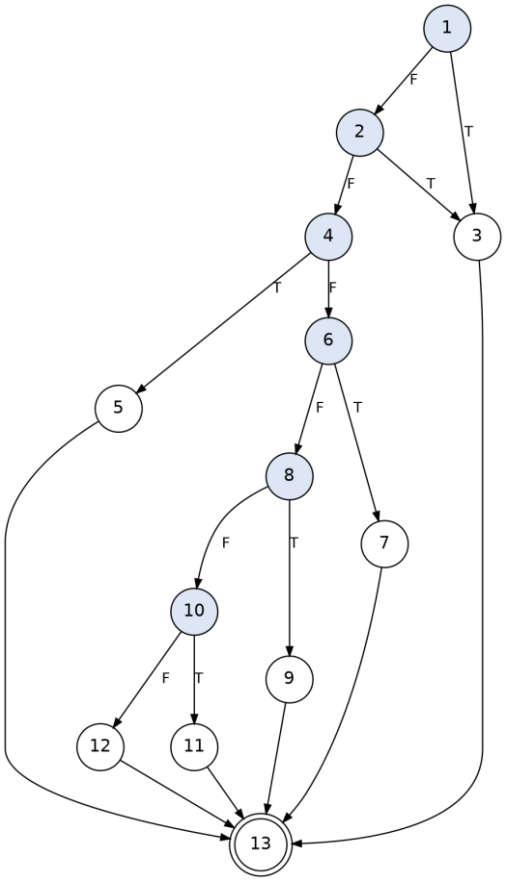
```
    void emailNull() {
        assertEquals("Email tidak valid",
                service.validasiPendaftaran("Budi", 20, null, true));
    }
}
```

Soal 4: Unit Test untuk StudentGrade

1. Cyclomatic Complexity calculateGrade()

Pemetaan node (kondisi majemuk dipecah, sama seperti konvensi pada Tugas Minggu 3):

Node	Isi
1	P1a: score < 0
2	P1b: score > 100
3	throw IllegalArgumentException
4	P2: score >= 80
5	return "A"
6	P3: score >= 70
7	return "B"
8	P4: score >= 60
9	return "C"
10	P5: score >= 50
11	return "D"
12	return "E"
13	Exit (node akhir)



Gambar 1. Flow graph method calculateGrade

Keterangan: lingkaran biru = node predikat; lingkaran ganda = node akhir; T = true; F = false.

Diketahui: E = 18, N = 13, P = 6.

Rumus	Perhitungan	Hasil
V(G) = E − N + 2	18 − 13 + 2	7
V(G) = P + 1	6 + 1	7
V(G) = jumlah region	6 region tertutup + 1 region luar	7

Kesimpulan: V(G) = 7. Catatan: jika score < 0 || score > 100 dianggap satu predikat, maka P = 5 dan V(G) = 6.

2. Independent Path calculateGrade()

Path	Jalur	Kondisi	Hasil
1	1-3-13	score < 0	Exception
2	1-2-3-13	score > 100	Exception
3	1-2-4-5-13	80 <= score <= 100	"A"
4	1-2-4-6-7-13	70 <= score < 80	"B"
5	1-2-4-6-8-9-13	60 <= score < 70	"C"
6	1-2-4-6-8-10-11-13	50 <= score < 60	"D"
7	1-2-4-6-8-10-12-13	0 <= score < 50	"E"

3. Unit Test JUnit untuk calculateGrade() dan isPassed()

Test case per independent path dilengkapi dengan nilai batas (boundary): 49/50, 59/60, 69/70, 79/80, serta 0, 100, −1, dan 101.

Path	score	calculateGrade()	isPassed()
1	-1	Exception	Exception
2	101	Exception	Exception
3	85 (batas: 80, 100)	"A"	true
4	75 (batas: 70, 79)	"B"	true
5	65 (batas: 60, 69)	"C"	true (60 adalah batas lulus)
6	55 (batas: 50, 59)	"D"	false (59 adalah batas tidak lulus)
7	25 (batas: 0, 49)	"E"	false

Kode unit test (StudentGradeTest.java):

```
import org.junit.jupiter.api.BeforeEach;
```

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.ValueSource;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.junit.jupiter.api.Assertions.assertTrue;

class StudentGradeTest {

    private StudentGrade studentGrade;

    @BeforeEach
    void setUp() {
        studentGrade = new StudentGrade();
    }

    // ==================== calculateGrade() ====================

    @ParameterizedTest(name = "score {0} -> grade {1}")
    @CsvSource({
            "100, A", "85, A", "80, A",
            "79, B",  "75, B", "70, B",
            "69, C",  "65, C", "60, C",
            "59, D",  "55, D", "50, D",
            "49, E",  "25, E", "0, E"
    })
    void calculateGrade_nilaiValid(int score, String expectedGrade) {
        assertEquals(expectedGrade, studentGrade.calculateGrade(score));
    }

    @ParameterizedTest(name = "score {0} harus melempar exception")
    @ValueSource(ints = {-1, 101, -100, 1000})
    void calculateGrade_nilaiTidakValid(int score) {
        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> studentGrade.calculateGrade(score));
        assertEquals("Score must be between 0 and 100", ex.getMessage());
    }

    // ======================== isPassed() ========================

    @Test
    void isPassed_lulusPadaBatas60() {
        assertTrue(studentGrade.isPassed(60));
    }

    @Test
    void isPassed_tidakLulusDi59() {
        assertFalse(studentGrade.isPassed(59));
    }

    @ParameterizedTest(name = "score {0} -> lulus = {1}")
```

```
    @CsvSource({"100, true", "75, true", "60, true",
            "59, false", "30, false", "0, false"})
    void isPassed_nilaiValid(int score, boolean expected) {
        assertEquals(expected, studentGrade.isPassed(score));
    }

    @ParameterizedTest(name = "score {0} harus melempar exception")
    @ValueSource(ints = {-1, 101})
    void isPassed_nilaiTidakValid(int score) {
        assertThrows(IllegalArgumentException.class,
            () -> studentGrade.isPassed(score));
    }
}
```

BAGIAN C: GITHUB ACTIONS

Soal 5: Analisis Workflow YAML

1. Nama workflow

Nama workflow adalah "Python CI" (ditentukan oleh baris name: Python CI).

2. Trigger/event yang memicu workflow

Event	Penjelasan
push	Workflow berjalan setiap ada push ke branch main atau develop.
pull_request	Workflow berjalan saat pull request dibuat atau diperbarui dengan branch tujuan (target) main.

Push atau pull request ke branch lain (misalnya feature-x) tidak memicu workflow ini.

3. Fungsi strategy matrix

Strategy matrix membuat satu job yang sama dijalankan berulang kali dengan variasi nilai variabel tertentu. Pada workflow ini variabelnya adalah python-version dengan nilai 3.10, 3.11, dan 3.12, sehingga job test dijalankan sebagai tiga job terpisah secara paralel. Nilainya dipakai melalui ekspresi ${{ matrix.python-version }} pada step Setup Python. Tujuannya memastikan kode dan test lolos di semua versi Python yang didukung (uji kompatibilitas) tanpa menulis tiga job manual, sehingga lebih ringkas dan lebih cepat.

4. Steps di dalam job test

No	Nama Step	Fungsi
1	Checkout code	Mengambil kode repository ke runner (actions/checkout@v4).
2	Setup Python ${{ matrix.python-version }}	Memasang versi Python sesuai nilai matrix (actions/setup-python@v5).
3	Install dependencies	Meng-upgrade pip, lalu memasang pytest dan pytest-cov.
4	Run tests	Menjalankan pytest dengan pengukuran coverage untuk folder src dan menghasilkan laporan coverage.xml.

Catatan: soal meminta 5 steps, sedangkan workflow di soal hanya memuat 4 step (daftar di atas sudah lengkap). Jika dosen mengharapkan 5 item, kemungkinan itu terjadi karena soal sengaja atau tidak sengaja menghitung pengaturan matrix atau job sebagai satu langkah tambahan.

Soal 6: Workflow YAML untuk Project Java

File: .github/workflows/java-unit-test.yml

```
name: Java Unit Test

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '17'
          cache: maven

      - name: Run tests
        run: mvn test

      - name: Generate coverage report
        run: mvn jacoco:report

      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/
```

Penjelasan singkat: workflow berjalan pada push ke main/develop dan pull request ke main, memakai ubuntu-latest, memasang JDK 17 (Temurin) dengan cache Maven, menjalankan mvn test, membuat laporan JaCoCo dengan mvn jacoco:report, lalu mengunggah folder laporan sebagai artifact bernama "jacoco-report".

Syarat: pom.xml harus memuat plugin jacoco-maven-plugin dengan goal prepare-agent (lihat pom.xml pada Bagian E) agar file jacoco.exec dihasilkan saat mvn test, sehingga mvn jacoco:report memiliki data untuk dibuat laporan.

BAGIAN D: CODE COVERAGE & INTEGRASI

Soal 7: Analisis Code Coverage

File	Stmts	Miss	Cover
src/calculator.py	10	0	100%
src/login_system.py	15	2	87%
src/payment.py	20	8	60%
src/utils.py	5	1	80%
TOTAL	50	11	78%

1. Total statement

10 + 15 + 20 + 5 = 50 statement.

2. Total statement yang tidak tereksekusi (Miss)

0 + 2 + 8 + 1 = 11 statement.

3. File dengan coverage terendah

File src/payment.py dengan coverage 60% (12 dari 20 statement tereksekusi, 8 statement belum).

4. Cara meningkatkan coverage payment.py

1. Jalankan pytest dengan opsi --cov-report=term-missing agar nomor baris yang belum tereksekusi (8 statement) terlihat jelas.
2. Tulis test untuk baris-baris tersebut, umumnya jalur yang jarang dilewati: penanganan error (pembayaran gagal, saldo tidak cukup, metode pembayaran tidak valid, jumlah nol atau negatif) dan cabang if/else yang belum diuji.
3. Tambahkan test nilai batas serta test negatif (input tidak valid) yang memverifikasi exception atau pesan error.
4. Gunakan mock/stub untuk dependensi eksternal (gateway pembayaran, database) agar jalur gagal dapat disimulasikan.
5. Jalankan ulang dan ulangi sampai target tercapai. Menutup 4 statement lagi sudah menaikkan payment.py menjadi 80% (16/20), dan menutup 6 statement menjadi 90% (18/20).

5. Apakah coverage 78% sudah cukup baik?

Menurut saya 78% adalah titik awal yang cukup baik, tetapi belum memadai. Alasannya:

- **Mendekati standar umum:** banyak tim menargetkan 70–80% sebagai batas minimum, sehingga 78% sudah di kisaran tersebut.
- **Tidak merata:** payment.py, yang kemungkinan menangani transaksi keuangan (modul kritis), hanya 60% dengan 8 statement tak teruji, padahal bug di modul ini paling berisiko.
- **Coverage hanya mengukur kuantitas:** baris yang tereksekusi belum tentu diverifikasi dengan assertion yang benar, sehingga angka tinggi tidak menjamin kualitas test.
- **Rekomendasi:** tetapkan ambang batas per file (modul kritis 90% atau lebih), fokus pada cabang (branch coverage), dan pantau melalui GitHub Actions agar coverage tidak turun.

BAGIAN E: STUDI KASUS TERPADU (BONUS)

Soal 8: Project BankAccount

1. Struktur Project

```
bank-account/
├── .github/
│   └── workflows/
│       └── java-ci.yml
├── src/
│   ├── main/java/BankAccount.java
│   └── test/java/BankAccountTest.java
├── .gitignore
├── pom.xml
└── README.md
```

2. Kode Program: BankAccount.java

Aturan validasi: amount tidak boleh negatif (deposit dan withdraw), dan withdraw tidak boleh melebihi saldo. Pelanggaran melempar IllegalArgumentException.

```
public class BankAccount {

    private double balance;

    public BankAccount() {
        this.balance = 0;
    }

    public BankAccount(double initialBalance) {
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Saldo awal tidak boleh negatif");
        }
        this.balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("Jumlah deposit tidak boleh negatif");
        }
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("Jumlah penarikan tidak boleh negatif");
        }
        if (amount > balance) {
            throw new IllegalArgumentException("Saldo tidak mencukupi");
        }
        balance -= amount;
    }

    public double getBalance() {
        return balance;
```

```
    }
}
```

3. Unit Test: BankAccountTest.java (10 test case)

No	Skenario	Jenis
1	Saldo awal default = 0	Normal
2	Saldo awal negatif melempar exception	Negatif
3	Deposit menambah saldo	Normal
4	Deposit beberapa kali diakumulasi	Normal
5	Deposit negatif melempar exception	Negatif
6	Withdraw mengurangi saldo	Normal
7	Withdraw sebesar saldo menghasilkan 0	Nilai batas
8	Withdraw melebihi saldo (100000.01) melempar exception	Nilai batas / negatif
9	Withdraw negatif melempar exception	Negatif
10	Deposit lalu withdraw menghasilkan saldo benar	Kombinasi

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

class BankAccountTest {

    private static final double DELTA = 0.001;
    private BankAccount account;

    @BeforeEach
    void setUp() {
        account = new BankAccount(100000);
    }

    @Test
    @DisplayName("Saldo awal default adalah 0")
    void saldoAwalDefault() {
        assertEquals(0, new BankAccount().getBalance(), DELTA);
    }

    @Test
    @DisplayName("Konstruktor dengan saldo awal negatif melempar exception")
    void saldoAwalNegatif() {
        assertThrows(IllegalArgumentException.class,
                () -> new BankAccount(-1));
    }
```

```
@Test
@DisplayName("Deposit menambah saldo")
void depositMenambahSaldo() {
    account.deposit(50000);
    assertEquals(150000, account.getBalance(), DELTA);
}

@Test
@DisplayName("Deposit beberapa kali diakumulasi")
void depositBeberapaKali() {
    account.deposit(10000);
    account.deposit(20000.5);
    assertEquals(130000.5, account.getBalance(), DELTA);
}

@Test
@DisplayName("Deposit negatif melempar exception")
void depositNegatif() {
    IllegalArgumentException ex = assertThrows(
            IllegalArgumentException.class, () -> account.deposit(-1));
    assertEquals("Jumlah deposit tidak boleh negatif", ex.getMessage());
    assertEquals(100000, account.getBalance(), DELTA);
}

@Test
@DisplayName("Withdraw mengurangi saldo")
void withdrawMengurangiSaldo() {
    account.withdraw(40000);
    assertEquals(60000, account.getBalance(), DELTA);
}

@Test
@DisplayName("Withdraw sebesar saldo menghasilkan saldo 0 (batas)")
void withdrawSeluruhSaldo() {
    account.withdraw(100000);
    assertEquals(0, account.getBalance(), DELTA);
}

@Test
@DisplayName("Withdraw melebihi saldo melempar exception")
void withdrawMelebihiSaldo() {
    IllegalArgumentException ex = assertThrows(
            IllegalArgumentException.class,
            () -> account.withdraw(100000.01));
    assertEquals("Saldo tidak mencukupi", ex.getMessage());
    assertEquals(100000, account.getBalance(), DELTA);
}

@Test
@DisplayName("Withdraw negatif melempar exception")
void withdrawNegatif() {
    assertThrows(IllegalArgumentException.class,
            () -> account.withdraw(-500));
}
```

```
    @Test
    @DisplayName("Deposit lalu withdraw menghasilkan saldo yang benar")
    void depositLaluWithdraw() {
        account.deposit(25000);
        account.withdraw(75000);
        assertEquals(50000, account.getBalance(), DELTA);
    }
}
```

4. pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>bank-account</artifactId>
  <version>1.0</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.2.5</version>
      </plugin>
      <plugin>
        <groupId>org.jacoco</groupId>
        <artifactId>jacoco-maven-plugin</artifactId>
        <version>0.8.12</version>
        <executions>
          <execution>
            <goals>
              <goal>prepare-agent</goal>
            </goals>
          </execution>
        </executions>
```

```
      </plugin>
    </plugins>
  </build>
</project>
```

5. GitHub Actions Workflow: .github/workflows/java-ci.yml

```
name: BankAccount CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '17'
          cache: maven

      - name: Run unit tests
        run: mvn test

      - name: Generate JaCoCo coverage report
        run: mvn jacoco:report

      - name: Upload coverage report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/
```

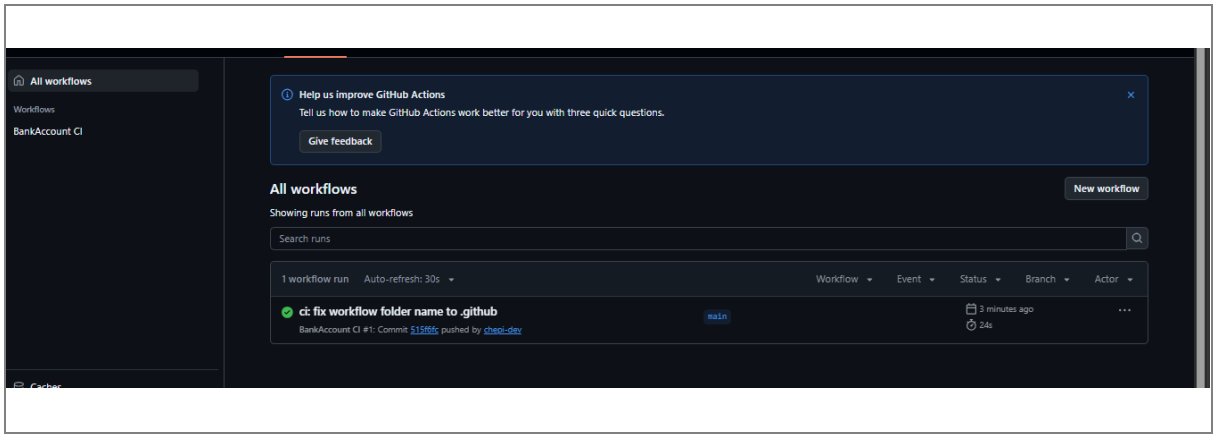
6. Langkah Upload ke GitHub

```
cd bank-account
printf "target/\n.idea/\n*.iml\n.vscode/\n" > .gitignore
git init
git add .
git commit -m "feat: add BankAccount with unit tests and CI"
git branch -M main
git remote add origin https://github.com/[USERNAME]/bank-account.git
git push -u origin main
```

Setelah push, buka tab Actions pada repository dan pastikan workflow berstatus hijau (sukses). Laporan coverage dapat diunduh dari bagian Artifacts pada run tersebut (jacoco-report), lalu dibuka melalui index.html.

7. Hasil: Screenshot Pipeline dan Coverage

Screenshot pipeline berjalan sukses (tab Actions):



Screenshot coverage report (JaCoCo index.html):



LINK REPOSITORY GITHUB

Keterangan	Link
Repository latihan Git (Bagian A, Soal 2)	chepi-dev/latihan-git-1224022
Repository project BankAccount (Bagian E)	chepi-dev/Project_BankAccount
Folder pengumpulan tugas	https://github.com/[USERNAME]/[REPO-TUGAS]/tree/main/Tugas/Tugas04